HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

PROJECT II

Design of the Automated Guided Vehicle control system

NGUYEN TRONG NGHIA
nghia.nt222796@sis.hust.edu.vn

Advanced Program in Control and Automation Engineering

Supervisor:	Dr. Hoang Duc Chinh	Supervisor's Signature
Department:	Automation	
School:	Electrical and Electronic Engineering	

HANOI, 02/2026

Acknowlegdement

I would like to express my sincere thanks to Dr. Hoang Duc Chinh for his valueable guidances, insightful feedback and continuous support throughout the development of this project.

I an also thankful to my project teammates, Nguyen Quoc Huy, Nguyen Trung Hieu and Dinh Hoang Son. Their cooperation, dedication and contribution play a crucial role in the process of the work. The completion of this project would not have been possible without their support.

Finally, I would like to thank the Faculty of Electrical and Electronics Engineering at Hanoi University of Science and Technology for providing the learning environment and resources that made this project possible.

Student

Full name and Signature

Abstract

This thesis presents the design and implementation of a centralized Master Control System for AGV fleets, strictly adhering to the VDA 5050 industrial standard to ensure interoperability. The core contribution is the development of a dynamic task allocation framework based on the Sequential Single-Item (SSI) Auction algorithm, which optimizes assignments by balancing Energy Consumption and Total Flow Time. Built upon a scalable architecture using Django, Docker, and MQTT, the system introduces a Marginal Cost logic with Task Chaining capabilities, allowing active vehicles to bid for future tasks based on predicted trajectories. Experimental validation on a simulated fleet confirms the system's correctness in protocol compliance and its effectiveness in enhancing fleet efficiency under dynamic constraints.

TABLE OF CONTENT

CHAPTER 1. INTRODUCTION 1
1.1 Introduction to Automated Guided Vehicle 1
1.1.1 History of AGV 1
1.1.2 Types of AGVs 1
1.1.3 AGV Navigation Methods 2
1.2 Problem Statement 3
1.3 Standardization in AGV Communication 4
1.4 Scope of work 5
CHAPTER 2. METHODOLOGY 6
2.1 Conceptual Overview 6
2.2 Environment Modeling 6
2.2.1 Map Model 6
2.2.2 Reservation Model 8
2.3 Objective Funtion 9
2.3.1 Energy Cost Model 10
2.3.2 Time Cost Model 11
2.4 Auction Algorithm 12
2.4.1 SSI Auction Workflow 12
2.4.2 Baseline Calculation 14
2.4.3 Marginal Cost Calculation and Task Chaining 15
2.4.4 Constraints Handling 17
2.4.5 Winner Selection 18
CHAPTER 3. SOFTWARE DESIGN 19
3.1 System Overview 19
3.2 Communication Protocol Design 20
3.2.1 MQTT Topic Namespace 20
3.2.2 Message Payload Specification 21
3.2.3 Communication Sequence Logic 23
3.3 Database Design 23
3.4 Software Module Design 25
3.4.1 Task Manager Module 26
3.4.2 Bidding Engine Module 26
3.4.3 Graph Engine Module 27

3.4.4 Physics Calculation Module 27
3.5 Application Programming Interface (API) Design 27
CHAPTER 4. EXPERIMENT AND RESULTS 30
4.1 Experimental Setup 30
4.2 Functional Verification 30
4.2.1 Task Assignment Logic 30
4.2.2 Task Chaining Capability 31
4.3 Performance Evaluation 31
4.3.1 Load Balancing 31
4.3.2 Execution Efficiency 32
4.4 Discussion 32
CHAPTER 5. CONCLUSION 33
5.1 Conclusion 33
5.2 Future Work 33
REFERENCES 34

LIST OF FIGURE

Figure 1.1 One of the first American AGVS (Source: Barrett Cravens/Savant Automation (1958)) 1
Figure 1.2 Types of AGV 2
Figure 1.3 AGV Navigation Methods 3
Figure 1.4 The goal of VDA 5050 is to enable every compliant automated guided vehicle (AGV) to work together via one fleet management software (left), rather than different master controllers being needed for each brand of AGV working on-site (right). 4
Figure 2.1 Graph representation including Base (solid lines) and Horizon (dashed lines) according to VDA 5050 7
Figure 2.2 The dynamic mechanism of updating the Base and Horizon during task execution 8
Figure 2.3 Time-Window Reservation Model illustrating conflict detection and resolution 9
Figure 2.4 Calculation Path for Idle AGV 16
Figure 2.5 Calculation Path for Busy AGV 17
Figure 3.1 System high-level architecture 19
Figure 3.2 Web functionality diagram 20
Figure 3.3 Structure of the Information Flow 23
Figure 3.4 Entity-Relationship Diagram of the Master Control System 24
Figure 3.5 UML Class Diagram 26
Figure 3.6 Overview of the Master Control System's API endpoints displayed in Swagger UI. 28
Figure 3.7 Detailed schema of the 'Create Order' API. The JSON payload requires specific topological node IDs to initiate the auction process. 29
Figure 4.1 List of simulation AGVs and its initial states 30
Figure 4.2 Log snippet showing successful winner determination 31
Figure 4.3 List of simulation Orders 31
Figure 4.4 Log of tasks each AGV did in the simulation 32

LIST OF TABLE

Table 3.1 MQTT Topic Definitions 21
Table 3.2 Common Header Structure 21
Table 3.3 Order Message Body 22
Table 3.4 State Message Body 22
Table 3.5 AGV metadata table 24
Table 3.6 Order data table 24
Table 3.7 AGV States table 25
Table 3.8 User table 25
Table 4.1 Task distribution among the fleet 31

CHAPTER 1. INTRODUCTION

1.1 Introduction to Automated Guided Vehicle

Automated Guided Vehicle (AGV) is autonomous mobile robots that can navigate and follows predefined paths. It is designed to transport materials in manufacturing facilities, warehouses and factories. AGVs enhance operational efficiency, costs, and improve workplace safety. Their integration into modern industrial systems plays a vital part in the shift towards automation and smart manufacturing. It is not only a key factor of modern industry but also involves more and more in human life in daily basis.

1.1.1 History of AGV

The first concept of AGV appeared back in 1953 when Arthur "Mac" Barrett launched the "Guide-O-Matic," - a modified towing tractor reconfigured to follow an overhead wire in a grocery. This innovation laid the foundation for subsequent advancements in automated material handling systems.

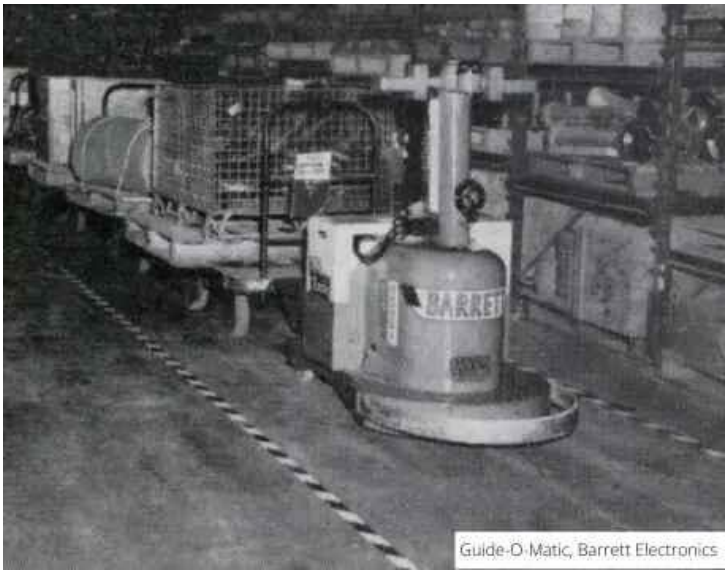


Figure 1.1 One of the first American AGVS (Source: Barrett Cravens/Savant Automation (1958))

1.1.2 Types of AGVs

AGVs can be categorized based on their design and functionality:

- Automated Guided Cart (AGC): Low-profile AGVs designed to carry loads placed on top of the vehicle (operate as under-ride systems with either lifting mechanisms or pin-hook attachments). Low cost, simple design, easy to deploy are the advantages of this AGV type.
- Tow Tractor AGV: Functioning like towing trucks, capable of pulling a train of carts or load units. They are generally faster and offer greater capacity compared to AGCs. This AGV type can also carry multiple loads simultaneously.

- Unit Load AGV: One of the most efficient AGV, using rollers on the vehicle to carry heavy loads (up to 20 tons). The standout benefits of this type of AGV are flexibility and workplace safety.
- Automated Guided Forklift: Can be considered as driverless forklift. This is the essential type of AGV in warehouses. Most of forklift AGV using laser navigation.

Each type of AGV has different functions and can be applied to operate a AGV system.

a. Automated Guided Cart

b. Tow Tractor AGV

c. Unit Load AGV

d. Automated Guided Forklift

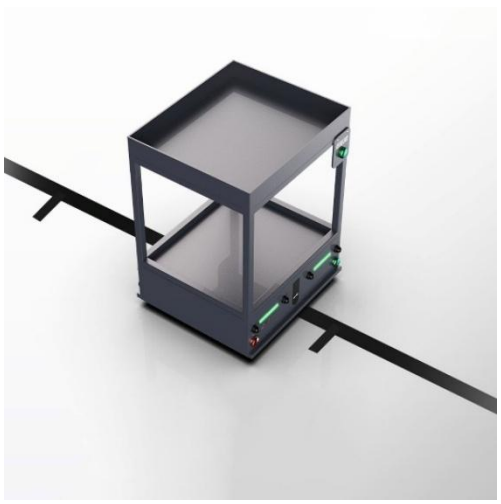
Figure 1.2 Types of AGV

1.1.3 AGV Navigation Methods

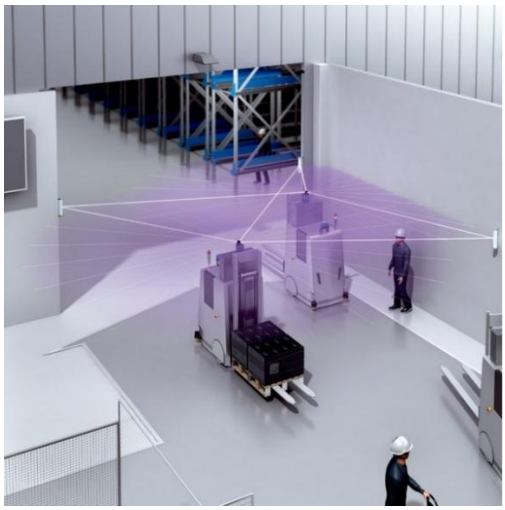
AGVs can be equipped with various navigation technologies depending on their level of autonomy, application scale and environment. The choice of navigation method significantly impacts system cost, complexity and flexibility. Some common methods include:

- Line following: One of the most basic and cost-effective methods. The AGV follows a visible black line (typically on a white surface) using infrared sensors. Suitable for structured indoor environments with fixed paths.

- Laser-Guided Navigation: AGVs use LIDAR or laser scanners to detect reflective targets or environmental features. Offers high precision and flexibility in path planning.



a. Line Following



b. Laser Guided Navigation

Figure 1.3 AGV Navigation Methods

1.2 Problem Statement

The manufacturing and logistics sectors are undergoing a significant transformation, driven by global market forces such as the just-in-time approach, mass customization, and e-commerce. This evolution is encapsulated by the Industry 4.0 paradigm, which emphasizes interconnectivity and flexibility to enhance productivity [1, 2]. In this context, smart factory environments demand effective object tracking and precise synchronization of internal logistics with production operations.

AGVs have emerged as a critical component in realizing smart logistics and flexible manufacturing systems (FMS). AGVs automate material transportation, particularly addressing the highly labor task of order picking, which can account for approximately 55% of total warehouse operating expenses [1]. The success of future production systems depends on deploying efficient AGV fleets for continuous and consistent material handling [1-3].

However, the effective management of multiple AGVs within a confined space presents significant challenges, particularly regarding control software, the progress of which has historically lagged behind hardware advancements. The complexity scales rapidly when attempting to operate a large fleet (e.g., hundreds of vehicles in modern applications like container handling) [3]. The critical operational issues can be categorized as follows:

- Efficiency and Task Allocation: Achieving high system throughput requires optimal dispatching and scheduling, minimizing the total travel time of vehicles [3]. Simple strategies like First-Come-First-Served (FCFS) or nearest vehicle rules often prove inadequate in maximizing system efficiency, especially with a high volume of tasks.

- Conflict and Congestion Management: AGVs operate in environments with limited path capacity where the length of the vehicle cannot be ignored, unlike traditional vehicle routing problems (VRP). This necessitates advanced algorithms to anticipate and eliminate hazards such as collisions, congestion, and deadlocks among competing vehicles vying for shared resources [3, 4].
- Scalability and Flexibility: Traditional centralized control architectures struggle to manage the computational load required for real-time routing and scheduling in large, dynamic environments, rendering the overall system inflexible and difficult to reconfigure in response to layout changes [2]. Most existing analytical solutions are only applicable to systems with a relatively small number of AGVs.

To address these limitations, there is an urgent requirement for an advanced control methodology that enables autonomous, decentralized decision-making and offers integrated management of task assignment and routing [2, 4]. This methodology must also support multi-objective optimization, moving beyond simple shortest-path metrics to include critical parameters such as energy consumption and total task duration.

1.3 Standardization in AGV Communication

To address the fragmentation caused by proprietary control systems, the German Association of the Automotive Industry (VDA) and the VDMA Mechanical Engineering Industry Association jointly developed the VDA 5050 standard. This interface specification defines a unified communication protocol between the Master Control and mobile robots (AGVs/AMRs) regardless of the manufacturer. By standardizing data exchange via MQTT (Message Queuing Telemetry Transport), VDA 5050 enables interoperability, allowing a single fleet management system to simultaneously orchestrate heterogeneous vehicles from different vendors within a shared environment [5].

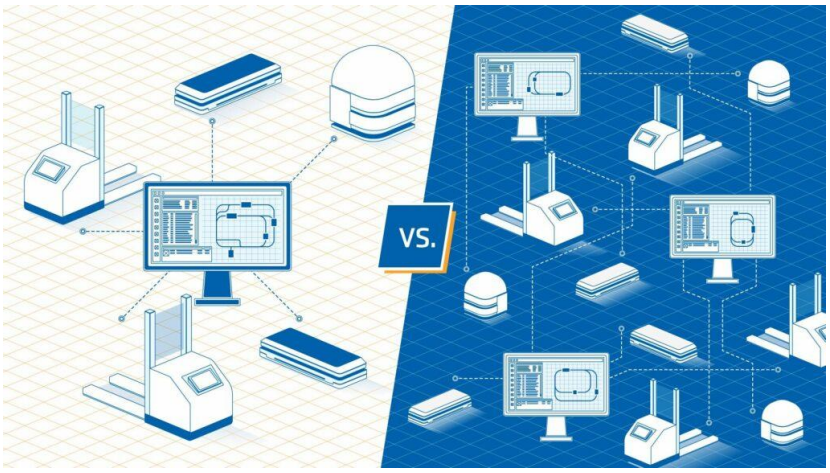


Figure 1.4 The goal of VDA 5050 is to enable every compliant automated guided vehicle (AGV) to work together via one fleet management software (left), rather than different master controllers being needed for each brand of AGV working on-site (right).

1.4 Scope of work

The scope of this thesis centers on the research, design, and implementation of a control system for AGVs within smart warehouse environments, strictly adhering to the VDA 5050 industrial standard. The project establishes a robust real-time communication infrastructure based on the MQTT protocol, defining and standardizing packet structures for Orders (topological paths and actions), States (real-time telemetry), and Instant Actions. This ensures interoperability and seamless data synchronization between the central server and a large fleet of robots.

Regarding algorithmic development, the project addresses the complex problem of task scheduling and allocation by modeling the environment as a Directed Graph and implementing the Sequential Single-Item (SSI) Auction mechanism. A key contribution of this work is the method that balances Energy Consumption and Total Flow Time (TFT). Moreover, the system implements solutions that can handle dynamic scenarios such as task chaining and queueing for active vehicles.

CHAPTER 2. METHODOLOGY

This chapter outlines in detail the algorithm architecture that governs the intelligent coordination of the AGV. The objective of this methodology is to assign and execute transportation tasks in an effective way by minimizing energy consumption, and total flow time (TFT). This framework is built upon three main algorithm: the Sequential Single-Item (SSI) auction for decentralized task allocation [6], Delegate Multi-Agent System (D-MAS) for dynamic environment modeling and route planning [6, 7], and an objective function that incorporates energy consumption and TFT.

2.1 Conceptual Overview

To manage the computational complexity of controlling multiple AGVs in a shared, resource-constrained environment, the coordination mechanism draws inspiration from the Delegate Multi-Agent System (D-MAS) paradigm. Unlike traditional centralized scheduling, which often suffers from computational bottlenecks when scaling, D-MAS mimics the self-organizing behavior of biological ant colonies (stigmergy) to explore the solution space efficiently [9], [10].

In this specific implementation, each physical AGV is represented by a Mobile Robotic System Agent (MRSA) residing within the Master Server's logic. As defined in D-MAS literature, the MRSA is a virtual entity mirroring its physical counterpart, responsible for making decisions and interacting with other agents [8]. Instead of calculating complex routes directly, these virtual agents interact with the environment through three specialized types of lightweight "Ant Agents" to gather information and reserve resources:

1. Feasibility Ant: These are simple, reactive agents deployed by Resource Agents (RAs) to roam the network and disseminate feasibility information [8]. They validate resource availability and collect information, esuring the static map accurately reflects resource capabilities [10].
2. Exploring Ant: These lightweight agents are issued by the MRSA (Task Agent) to explore potential paths and evaluate different routes. The exploring ant calculates the realistic cost of a candidate path by querying a dynamic reservation schedule maintained by Resource Agents. It reports the resulting schedule (including anticipated delays) back to the MRSA [7].
3. Intention Ant: Once an MRSA wins a task and selects an optimal route, it sends out an Intention Ant to formalized the plan. This ant reserves required time slots in the environment [6, 9]. Intention Ants essentially inform the Resource Agents of the winning MRSA's intent to follow a path [11].

2.2 Environment Modeling

2.2.1 Map Model

To ensure interoperability and standardized communication between the Master Control and the AGVs, the physical warehouse environment is modeled as

a directed graph $G = (N, E)$, strictly adhering to the VDA 5050 interface specification. This graph structure defines the navigable space and logistical checkpoints required for task execution.

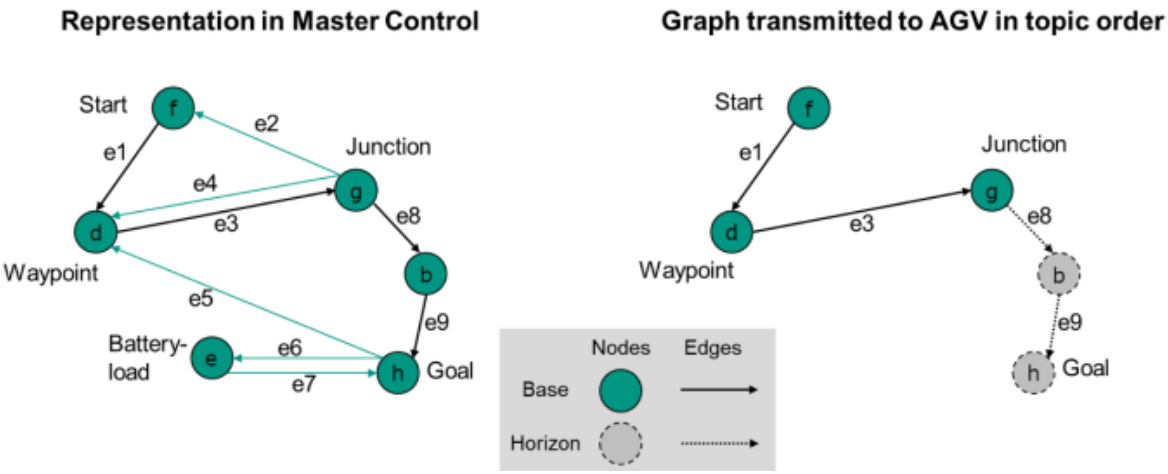


Figure 2.1 Graph representation including Base (solid lines) and Horizon (dashed lines) according to VDA 5050

Nodes (N) represent discrete points within the coordinate system (e.g., workstations, intersections, charging stations) or decision points for navigation. According to the VDA 5050 standard, each node $n \in N$ is characterized by the following attributes:[5]

- Node ID and Sequence ID: A unique identifier (nodeId) and a sequence number (sequenceId) to track the order of traversal
- Position: The physical coordinates (x, y) and the orientation (θ) relative to the global project-specific map coordinate system.
- Actions: A list of specific operations (e.g., lift, drop, wait) that the AGV must execute upon reaching the node.

Edges (E) represent the directional connections between two nodes (n_{start}, n_{end}). Each edge $e \in E$ dictates the path the AGV must follow and is defined by:[5]

- Edge ID: A unique identifier (edgeId) linking the start and end nodes.
- Parameters defining the movement restrictions, such as maximum speed (maxSpeed), orientation type (orientationType).
- Corridor: Optional boundaries defining the allowed deviation from the ideal trajectory to facilitate obstacle avoidance

Base and Horizon Concept To manage traffic effectively and handle network latency, the route graph in the transport order is dynamically segmented into two distinct parts based on the released attribute:[5]

- Base (Released Area): This segment comprises the set of Nodes and Edges where the attribute released is set to True. The Base represents the authorized path that the AGV is currently permitted to traverse. The last node of the Base is designated as the Decision Point. If no further update is received, the AGV must come to a stop at this point.
- Horizon (Planning Area): This segment comprises the set of Nodes and Edges where the attribute released is set to False. The Horizon represents

the planned future route (look-ahead plan). The AGV processes this information for path planning and reservation purposes but is restricted from physically entering this area until a formal Order Update is received converting the Horizon (or part of it) into the Base.

This "Base and Horizon" mechanism allows the coordination system to continuously update the route (re-routing) and reserve resources dynamically without stopping the vehicle, ensuring smooth transitions and deadlock avoidance.

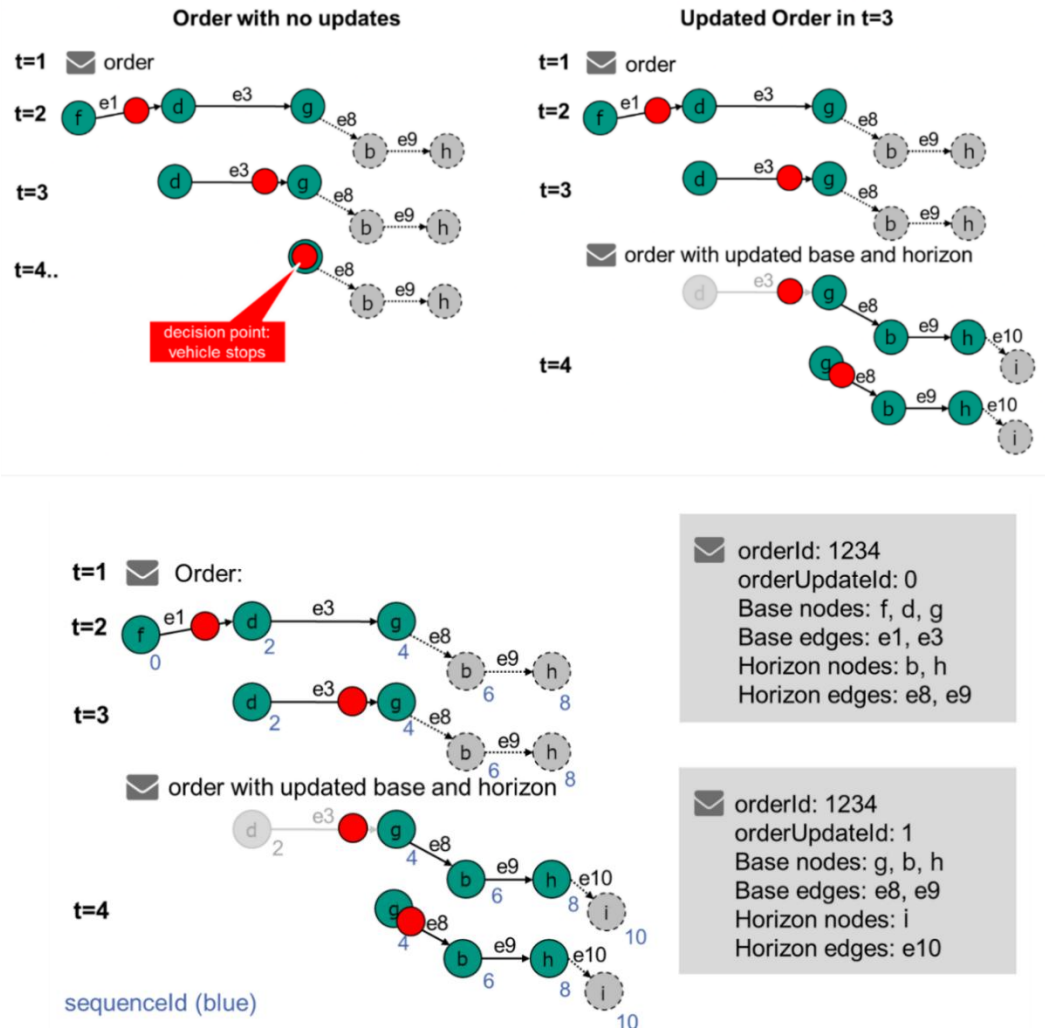


Figure 2.2 The dynamic mechanism of updating the Base and Horizon during task execution

2.2.2 Reservation Model

To manage congestion and ensure deadlock-free navigation without relying solely on decentralized negotiation, the Master Control maintains a centralized Dynamic Reservation Table. The environment is modeled as a set of shared resources (Nodes and Edges) with time-dependent availability, treating the layout as a set of resources that can be allocated in the time dimension [6]. A reservation entry is formally defined as a tuple:

$$R = \{r_{id}, t_{entry}, t_{exit}, agv_{id}, status).$$

Where:

- r_{id}: The unique identifier of the resource (Node or Edge).
- $[t_{entry}, t_{exit}]$: The specific time window during which the resource is exclusively occupied by the vehicle [8].
- agv_id: The identifier of the vehicle holding the reservation.
- $status$: Indicates the type of reservation (e.g., Hard for Base, Soft for Horizon).

When the Path Planning algorithm evaluates a potential route, it queries the Reservation Table for availability. A conflict is detected if the requested time interval $[t_{entry}, t_{exit}]$ for the same resource r_{id}:

$$[t_{start}, t_{end}] \cap [t_{entry}, t_{exit}] \neq \emptyset$$

If a conflict is detected, the algorithm does not discard the route immediately. Instead, it calculates a Wait Cost (Delay) required to resolve the conflict. The entry time is shifted to $t_{new_entry} = \max(t_{start}, t_{exit})$, ensuring the vehicle waits until the resource is free [6]. This mechanism effectively converts spatial conflicts into temporal delays, allowing the optimizer to trade off between waiting and re-routing.

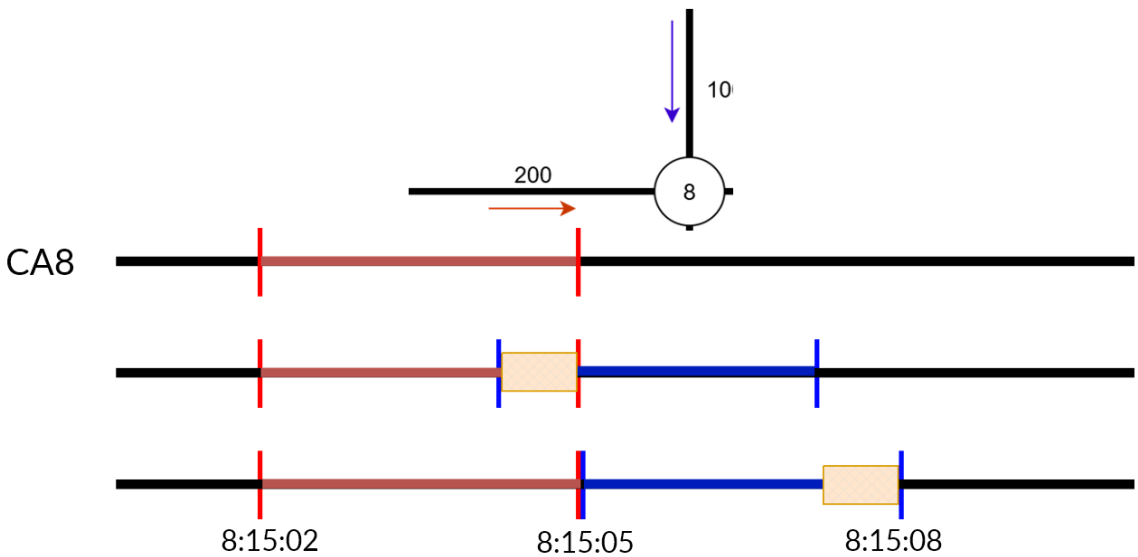


Figure 2.3 Time-Window Reservation Model illustrating conflict detection and resolution

This model directly supports the VDA 5050 logic by strictly enforcing the Base and Horizon concepts in the space-time domain:

- Base (Hard Reservation): Resources assigned to the Base are booked with a "Hard" status, guaranteeing exclusive access and preventing other agents from planning overlapping trajectories.
- Horizon (Soft Reservation): Resources in the Horizon are booked with a "Soft" status. These serve as a look-ahead plan to inform other agents of future intentions but can be overridden or rescheduled if higher-priority conflicts arise [6], [8].

2.3 Objective Funtion

The system minimizes the multi-objective function J, defined as a weighted combination of time and energy costs, implicitly including resource and routing constraints [6].

2.3.1 Energy Cost Model

Energy consumption is a critical factor regarding the operational efficiency of the AGV fleet. This project employs a dynamic energy model that accounts for the physical forces acting on the vehicle during translational movement (moving straight edges) and rotational movement (turning at nodes). This approach enables the bidding system to distinguish the efficient of paths.

The total electrical energy (E_{total}) required for a task is calculated by integrating the electrical power (P_e) over the travel time t_{run}. The electrical power is derived from the mechanical power (P_m) and the motor efficiency η_{motor} [12]:

$$E_{total} = \int_0^{t_{run}} P_e(t)dt \approx \sum P_e \cdot \Delta t \qquad (2.1)$$

$$P_e = \frac{P_m}{\eta_{motor}} = \frac{(P_{m,trans} + P_{m,rot})}{\eta_{motor}} \qquad (2.2)$$

Where:

- $P_{m,trans}$: Mechanical power required for translational movement
- $P_{m,rot}$: Mechanical power required for rotational movement

a) Translational Energy

The power required to propel the AGV forward must overcome the rolling friction resistance (F_r) and the acceleration resistance (F_a). According to Newton's second law and the motion dynamics presented in [12], the translational power is formulated as:

$$P_{m,trans} = (F_r + F_a) \cdot v_{AGV} \qquad (2.3)$$

$$P_{m,trans} = (m_{agv} + m_{load}) \cdot (\mu_r g + a) \cdot v_{AGV} \qquad (2.4)$$

Where:

- m_{agv}: Mass of the vehicle (kg)
- m_{load}: Mass of the carried payload (kg)
- $\mu_r g$: Rolling friction coefficient
- a: Acceleration of the AGV (m/s^2)
- v_{AGV}: Instantaneous velocity (m/s)
- g: Gravitational acceleration

This formula highlights that carrying a heavy load (m_{Load}) of frequently changing velocity ($a > 0$) significantly increases energy consumption compared to cruising at a constant speed [12].

b) Rotational Energy

Since the AGV operates on a graph-based map with discrete nodes, it frequently performs rotation actions (typically 90° or 180°) to navigate intersections. The power required for rotation depends on the torque (T) needed to overcome friction across the wheelbase (l_{AGV}) [12]:

$$P_{m,rot} = \frac{T \cdot \omega_{AGV} \approx (m_{agv} + m_{load}) \cdot (\mu_r g + a) \cdot l_{AGV}}{2} \qquad (2.5)$$

Where:

- l_{AGV}: Distance between the controlled wheels (wheelbase)
- ω_{AGV}: Angular velocity (rad/s)

2.3.2 Time Cost Model

In dynamic scheduling, minimizing travel distance alone is insufficient to guarantee system efficiency. The system must optimize the Total Flow Time (TFT), which represents the complete duration from the moment a task is assigned until its completion. Unlike static path planning, this model incorporates the AGV's current state (idle or busy) and resource constraints to calculate a realistic completion time.

The total time cost (T_{total}) required for a specific AGV to execute a new assignment is composed of the time waiting to become available and the actual execution time of the task [13]:

$$T_{total} = T_{wait} + T_{travel} \qquad (2.6)$$

a) Waiting Time

This component represents the Availability cost. It ensures that the auction mechanism penalizes AGVs that are currently occupied or far from the pickup point, preventing the unbalancing phenomenon where tasks pile up on a single optimal vehicle.

- If the AGV is idle, T_{wait} is simply the time required to travel from its current position to the new task's Pickup Node (Deadheading time).
- If the AGV is busy, T_{wait} is the estimated time required to complete its current queue of orders plus the travel time from the last drop-off node to the new task's pickup node:

$$T_{wait} = \frac{d_{remain} + d_{deadhead}}{v_{agv}} \qquad (2.7)$$

Where d_{remain} is the distance of the current active route and $d_{deadhead}$ is the distance to the new task's start point.

b) Travel Time

This is the duration required to perform the specific legs of the new task (Pickup → Delivery). To maintain consistency with the Energy Model, T_{travel} should ideally account for the kinematic profile (acceleration, cruising, deceleration). However, for rapid bid calculation in the simulation environment, this is approximated using the average velocity (v_{avg}) and the path distance (d_{task}) [14]:

$$T_{travel} \approx \frac{d_{task}}{v_{avg}} + N_{turns} \cdot t_{turn_avg} \quad (2.8)$$

Where:

- d_{task}: Total distance of the new assignment
- v_{avg}: Average operational speed of the AGV
- N_{turns}: Number of rotation actions required at intersections

2.4 Auction Algorithm

To address the dynamic task allocation problem in a multi-AGV environment, this project implements the Sequential Single-Item (SSI) auction mechanism. Unlike combinatorial auctions that can be computationally prohibitive, the SSI framework offers an efficient trade-off between solution quality and computation time by allocating one task at a time through successive auction rounds [14], [15].

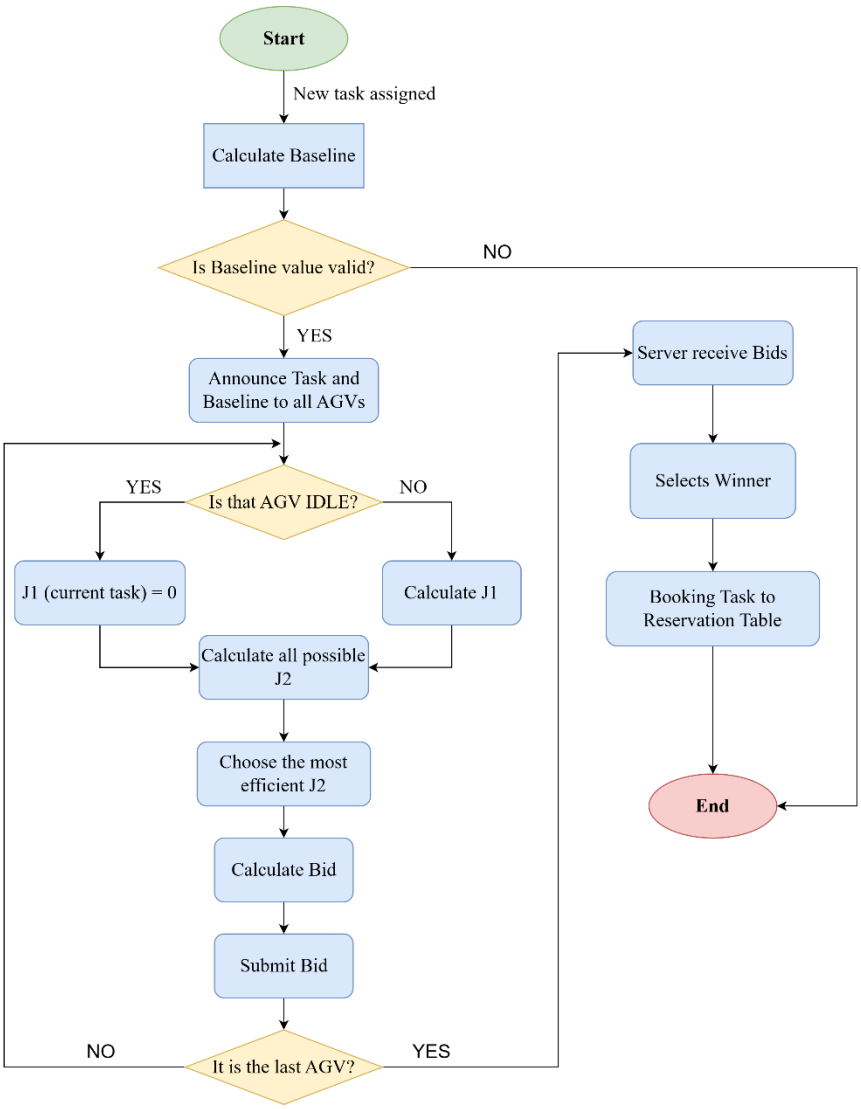
Although the system operates on a centralized Master Control architecture, the algorithm incorporates Multi-Agent System (MAS) principles to decentralize the cost calculation logic. The roles are defined as follows:

- Auctioneer: Responsible for announcing tasks, calculating the Baseline Cost for normalization, and executing the winner determination protocol.
- Bidders: These are software agents representing each physical AGV. Each bidder independently calculates its Marginal Cost based on its specific state (Current Position, Battery Level, Active Schedule) and local resource constraints.

The primary objective of this algorithm is to identify the optimal (Task, AGV) pairing that minimizes a Hybrid Objective Function, balancing both Energy Consumption and Total Flow Time (TFT).

2.4.1 SSI Auction Workflow

The auction process for a new transport task (T_{new}) follows a structured four-phase workflow: Initialization, Bidding, Selection, and Assignment.



a) Task Announcement & Baseline Calculation

Upon the creation of a new transport order (defined by $Node_{pickup}, Node_{delivery}$ and payload m_{load}), the Auctioneer first initiates the process. Before broadcasting, the system computes a Baseline Cost, this represents the ideal cost of performing the task under perfect conditions derived from the static graph. This value serves as a normalization factor, allowing the system to standardize raw cost values (Energy in kJ, Time in seconds) into a comparable, unitless score for the bidding process.

b) Marginal Cost Calculation

The Server broadcasts the auction request to all AGVs currently in the ONLINE state. Each eligible AGV calculates its Marginal Cost (b_{ms}) – defined as the incremental cost added to its existing schedule if it accepts the new task [14]:

$$J_{marginal} = J_{new_schedule} - J_{current_schedule} \qquad (2.9)$$

The calculation logic adapts to the AGV's operational state:

- Idle State: The marginal cost is the total cost of traveling from the AGV's current coordinates to the Pickup Node, and subsequently to the Delivery Node.

- Task Chaining: A key feature of this algorithm is its support for Task Chaining. An active AGV participates in the auction without stopping. The starting point for the new task is assumed to be the final node of its current active order ($lastNodeId$). The marginal cost accounts for the Wait Time (T_{wait}) required to finish the current job plus the cost of the new trajectory legs.

To ensure system stability, if an AGV's battery level is below the Critical Threshold, it automatically withdraws from the auction by submitting an infinite bid ($Bid = \infty$).

c) Winner Determination

The Auctioneer collects all valid bids ($Bid < \infty$) within a specific time window. Following the Lowest Bid (LB) strategy, the AGV with the lowest Bid Score is selected as the winner [14], [15]. This strategy ensures the system consistently moves towards a locally optimal state for every assignment decision, minimizing the total objective function (MiniSum) [15].

d) Order Dispatching

Once the winner is identified, the task status is updated to ASSIGNED, and the dispatch logic adheres to the VDA 5050 protocol.

2.4.2 Baseline Calculation

In multi-objective optimization problems involving heterogeneous metrics – specifically Energy Consumption (measured in kilojoules) and Time (measured in seconds) – direct summation of cost values is mathematically invalid due to unit incompatibility. To address this, the proposed methodology incorporates a Baseline calculation strategy to convert raw physical values into dimensionless scores, facilitating a fair and weighted comparison between competing agents.

a) Definition of Baseline Cost

The Baseline is defined as the theoretical cost of executing a specific task under ideal conditions. An ideal condition implies:

- The AGV travels along the shortest path determined by the Dijkstra algorithm on the static graph $G(N, E)$.
- The movement is continuous with zero wait time (no traffic congestion or obstacle delays).
- The path is traversed at the nominal operational speed without unexpected interruptions.

This ideal cost serves as the denominator for normalization. By comparing the actual Marginal Cost (which includes realistic waiting times and battery penalties) against this Baseline Cost, the system derives a standardized performance ratio.

b) Normalization Formulation

For a specific transport trajectory ($N_{start} \rightarrow N_{end}$) the system first computes the ideal physical metrics: Baseline Energy (E_{base}) and Baseline Time (T_{base}) based on the physical models described in Section 2.3.

The raw marginal cost of an AGV (E_{actual}, T_{actual}) are then normalized using the following formulations:

$$E_{norm} = \frac{E_{acual}}{E_{base}}; T_{norm} = \frac{T_{actual}}{T_{base}}$$

This approach ensures that the bid score reflects relative efficiency rather than absolute magnitude.

- If score ≈ 1.0: The AGV is operating at near-optimal efficiency
- If score $\gg 1.0$: The AGV is inefficent for this specific task

2.4.3 Marginal Cost Calculation and Task Chaining

The core of the SSI auction mechanism lies in the calculation of the Marginal Cost $(J_{Marginal})$. Unlike static routing, which considers only the absolute distance from a fixed origin, the marginal cost evaluates the incremental impact of inserting a new task into an AGV's existing schedule. This approach allows the system to implement an Insertion Heuristic, enabling busy vehicles to bid for future tasks based on their predicted location after completing current assignments [14], [15].

The marginal cost is formally defined as the difference between the total cost of the new schedule (with the task inserted) and the total cost of the current schedule. The

$$J_{marginal} = J(S_{new}) - J(S_{current}) \qquad (2.9)$$

Where $J(S)$ represents objective function value (weighted sum of Energy and Time) for a given schedule S. This logic is differentiating between two operational states:

a) AGV is idle

When an AGV is free, the starting point for the new task is simply its current physical coordinates. The marginal cost is equivalent to the total cost of performing the task, including the "deadhead" travel (moving empty to the pickup point).

- Start Node: $N_{start} = AGV_{current_position}$
- Wait time: $T_{wait} = 0$
- Marginal Energy:

$$E_{mar} = E(N_{start} \rightarrow N_{pickup}) + E(N_{pickup} \xrightarrow{load} N_{delivery})$$

- Marginal Time:

$$T_{mar} = T(N_{start} \rightarrow N_{pickup}) + T(N_{pickup} \xrightarrow{load} N_{delivery})$$

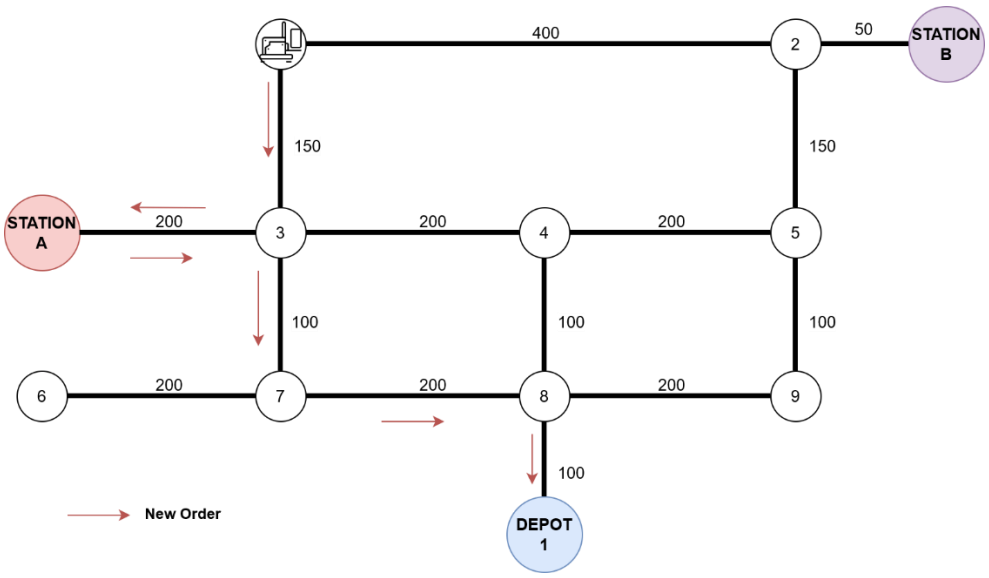


Figure 2.4 Calculation Path for Idle AGV

b) AGV is busy

This scenario addresses the Task Chaining capability. If an AGV is currently executing an order, it does not need to stop to participate in the auction. Instead, the system treats the new task as an extension to the AGV's Horizon (as defined in VDA 5050).

The system identifies the final destination of the current active order (specifically the last node in the Horizon) and treats it as the starting point for the new task.

$$N_{start} = Order_{current} \cdot LastNode$$

The system estimates the time required for the AGV to finish its current workload. This acts as a penalty for busy vehicles to prevent bottlenecks.

$$T_{wait} = \frac{d_{remain}}{v_{avg}}$$

The marginal energy considers only the energy consumed for the new trajectory segments (since the energy for the current task is a "sunk cost"). However, the marginal time must reflect the completion time of the new task, which includes the waiting time caused by the current task.

$$E_{mar} = E\left(N_{last_node} \rightarrow N_{pickup}\right) + E\left(N_{pickup} \xrightarrow{load} N_{delivery}\right)$$
$$T_{mar} = T\left(N_{last_node} \rightarrow N_{pickup}\right) + T\left(N_{pickup} \xrightarrow{load} N_{delivery}\right)$$

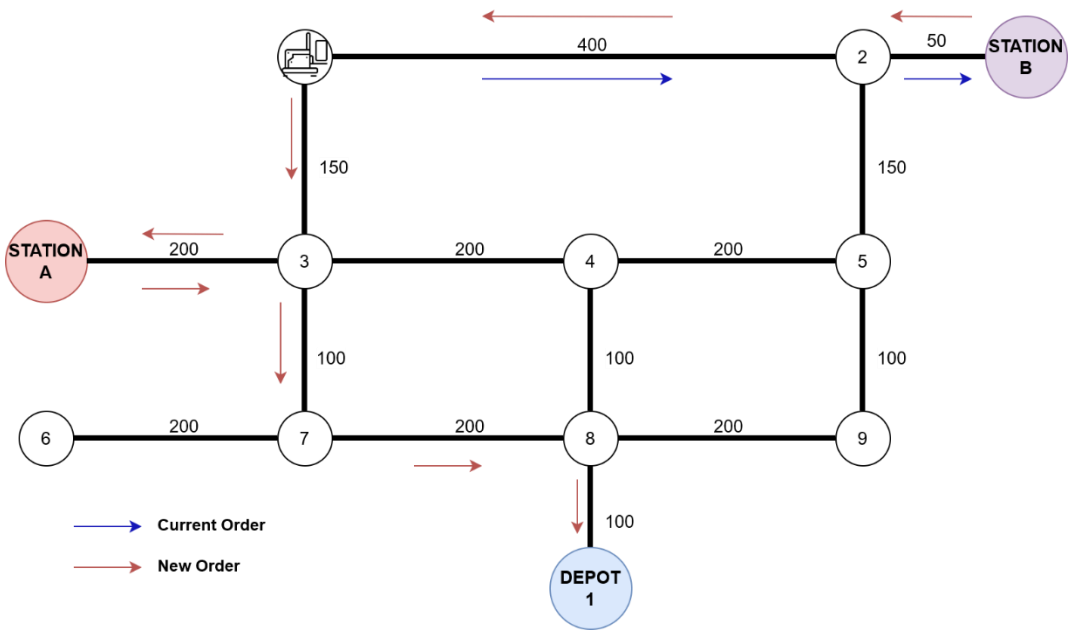


Figure 2.5 Calculation Path for Busy AGV

This logic allows the auctioneer to make globally efficient decisions aligned with the MiniSum objective (minimizing total fleet cost) [14]:

- A busy AGV finishing a task near the new pickup point may have a lower E_{mar} (due to short deadhead distance) than an idle AGV located far away, leading to reduced total energy consumption.
- By appending tasks to the Horizon, the AGV operates smoothly without intermittent stops between orders [5].

2.4.4 Constraints Handling

In practical industrial scenarios, minimizing the objective function (MiniSum) is not the sole criterion for assignment. The system must first validate the feasibility of the allocation to ensure operational safety and stability. The auction algorithm implements a two-layer validation logic consisting of Hard Constraints (resulting in immediate disqualification) and Soft Constraints (imposing a penalty on the bid score) [14].

a) Resource Constraints

To prevent AGVs from depleting their battery mid-task or becoming stranded, the bid calculator enforces a strict energy validation logic based on the vehicle's State of Charge (β):

- Hard Constraint: If the AGV's current battery level (β_{curr}) falls below the critical threshold ($\beta_{crit} = 10\%$), the AGV is deemed unfit for transport operations. To strictly enforce this, the system assigns an infinite cost to the bid, effectively disqualifying the AGV from the auction

$$If\ \beta_{curr} < \beta_{crit} \rightarrow Bid = \infty \text{ (Disqualified)}$$

- Soft Constraint: If the battery level is above the critical limit but below the safety threshold ($\beta_{safe} = 30\%$), the AGV is technically capable but should be discouraged from taking tasks unless it is the only viable option. Instead

of a hard rejection, the system applies a penalty factor (P_{bat}) to the final bid score:

$$If\ \beta_{crit} \leq \beta_{curr} < \beta_{safe} \rightarrow Bid_{final} = Bid_{raw} \times P_{bat}$$

Where $P_{bat} > 1.0$. This mechanism allows a low-battery AGV to win a task only if its topological advantage (very close to pickup) outweighs the energy penalty, ensuring flexibility while prioritizing fleet safety [16].

b) Routing Constraints

Before calculating detailed energy or time costs, the system validates the topological connectivity between the AGV's starting position and the target nodes on the directed graph $G(N, E)$.

If the path search algorithm (Dijkstra) returns an infinite cost ($Cost = \infty$), it indicates that the target is unreachable. In such cases, the AGV is automatically disqualified ($Bid = \infty$) to prevent execution errors and potential deadlocks [8].

2.4.5 Winner Selection

After collecting all valid bids from the available agents, the Auctioneer (Server) proceeds to the final phase of the SSI workflow: determining the optimal vehicle and translating the logical assignment into a physical command compliant with the VDA 5050 standard.

a) Objective Function

The core of the selection process is the Hybrid Objective Function, which aggregates the heterogeneous normalized metrics (Energy and Time) into a single scalar value ($Score$). To provide flexibility for different operational requirements (prioritizing speed during peak hours vs. energy saving during off-peak hours), the system applies configurable weight coefficients (ω_E, ω_T) [12], [14].

The final bid score for an AGV_i is calculated as follows:

$$Score_i = (\omega_E \cdot E^i_{norm} + \omega_T \cdot T^i_{norm}) \times P^i_{penalty} \qquad (2.10)$$

Where:

- ω_E, ω_T: Weight coefficients for Energy and Time, respectively (where $\omega_E + \omega_T = 1$).
- E^i_{norm}, T^i_{norm}: Normalized marginal costs calculated in Section 2.4.2
- $P^i_{penalty}$: Penalty factor derived from constraints

b) Winner Determination

The Auctioneer compares the final scores of all candidates. Following the standard Lowest Bid (LB) resolution rule [15], the AGV with the minimum score is declared the winner:

$$AGV_{winner} = arg \min_{i \in \{1 \ldots k\}} (Score_i)$$

This selection guarantees that the assigned vehicle represents the best local trade-off between proximity, efficiency, and system constraints at the moment of decision-making.

CHAPTER 3. SOFTWARE DESIGN

3.1 System Overview

The control system is designed based on a Centralized Master Control architecture integrated with decentralized decision-making capabilities. This hybrid approach facilitates seamless interaction between the operator, the control logic, and the physical AGV fleet while ensuring scalability and robustness. As illustrated in Figure 3.1, the high-level architecture comprises four main components: the AGV Fleet, the MQTT Broker, the Master Control Server, and the Web Client.

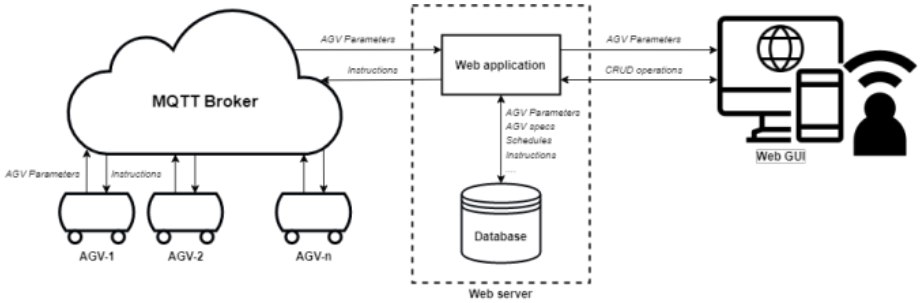


Figure 3.1 System high-level architecture

- Communication Layer: The communication backbone relies on the MQTT protocol over Wi-Fi, ensuring real-time data transmission. The MQTT Broker acts as the intermediary, routing VDA 5050 compliant messages: AGVs publish their real-time telemetry (Position, Battery, Status) to state topics, while the Server publishes transport commands to order topics [5].
- Control Layer: The Web Server (Master Control) is the core processing unit, encapsulated within Docker containers to ensure scalability and consistency. It hosts the Web Application (Backend logic) and a PostgreSQL Database for persistent storage of fleet data, map topology, and operational logs.
- Presentation Layer: The Web GUI communicates with the Server via RESTful APIs (HTTP/HTTPS), allowing operators to input tasks and monitor system performance visually.

Functionally, the Master Control Server is divided into distinct logic modules to handle specific responsibilities, as shown in Figure 3.2:

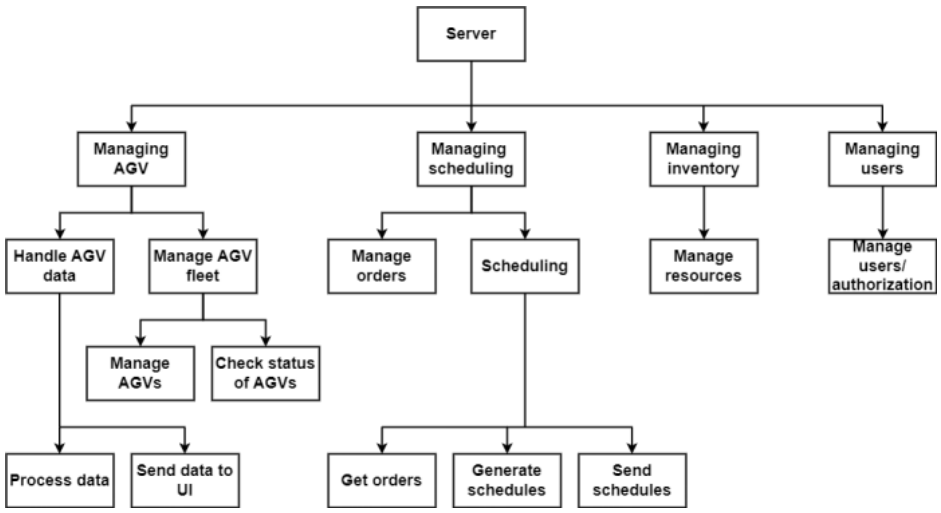


Figure 3.2 Web functionality diagram

- AGV Fleet Management: This module acts as the bridge between the physical and digital worlds. It handles the registration of vehicles based on the VDA 5050 factsheet and monitors their online/offline status via the connection topic. It parses incoming state messages to update the Digital Twin of the fleet in the database, ensuring the scheduler always has access to the latest vehicle status (Position, Battery, Load).
- Scheduling & Task Allocation: This is the intelligent core of the system. Unlike basic scheduling, this module implements the SSI Auction Algorithm (via the Bidding Engine). It processes incoming transport requests, calculates costs based on energy and time, and assigns the task to the optimal AGV.
- Order Management: Once a winner is selected, this function generates the specific topological path (Nodes and Edges) and manages the lifecycle of the order (Created → Active → Finished).
- User & Resource Management: This module secures the system by managing user authentication and provides APIs for external systems to interact with the server.

3.2 Communication Protocol Design

To ensure interoperability, scalability, and standardization, the system implements a communication interface strictly adhering to the VDA 5050 (Version 2.1.0) standard. This interface operates over the MQTT protocol , utilizing a Publish-Subscribe pattern to decouple the Master Control Server from the AGV fleet. Data payloads are encoded in JSON format, allowing for flexibility and future extensibility.

3.2.1 MQTT Topic Namespace

The topic structure is hierarchically designed to uniquely identify each agent within the fleet, preventing message collisions. Following the VDA 5050 recommendation, the namespace follows the pattern [5]:

{interfaceName}/{majorVersion}/{manufacturer}/{serialNumber}/{topic}

Table 3.1 MQTT Topic Definitions

Topic Suffix	Direction	QoS	Description
order	Server → AGV	0	Channel for transmitting transport orders (paths and actions). VDA 5050 recommends QoS 0 to reduce overhead, relying on state feedback for confirmation
state	AGV → Server	0	Channel for reporting real-time status (Position, Battery, Order Progress). Published periodically (e.g., 30s) or upon event triggers
instantActions	Server → AGV	0	Channel for immediate commands (e.g., Pause, Resume, Cancel) that must be executed independently of the current order context
connection	AGV → Server	1	Channel for the "Last Will and Testament" (LWT) message to detect online/offline status and unexpected disconnections
factsheet	AGV → Server	0	Channel where the AGV publishes its physical properties (geometry, speeds, load capabilities) to the master control

3.2.2 Message Payload Specification

All VDA 5050 messages share a common Header structure to ensure traceability and synchronization across the distributed system.

Table 3.2 Common Header Structure

Field	Data Type	Description
headerId	integer	Unique ID for the message, incremented per topic per message
timestamp	string	ISO 8601 timestamp (e.g., 2024-05-20T10:00:00Z)
version	string	Protocol version (2.1.0)
manufacturer	string	Manufacturer ID
serialNumber	string	Unique serial number of the AGV

a) Order Message Specification

The order message instructs the AGV to traverse a graph of Nodes and Edges. Crucially, the graph is divided into a Base (released for travel) and a Horizon (planned but not yet released)[5].

Table 3.3 Order Message Body

Field	Data Type	Description
orderId	string	Unique identifier for the transport mission
orderUpdateId	integer	Sequence number. Incremented when the existing order is extended (Horizon becomes Base) to allow continuous driving
nodes	array[Object]	List of Node objects. Each node contains nodeId, sequenceId, and actions
edges	array[Object]	List of Edge objects connecting the nodes. Contains constraints like max speed

b) State Message Specification

The state message is the heartbeat of the system, providing the Master Control with visibility into the AGV's operation and order progress.

Table 3.4 State Message Body

Field	Data Type	Description
orderId	string	The ID of the order currently being executed
orderUpdateId	integer	The update ID currently being processed, used to acknowledge order updates
lastNodeId	string	The ID of the last node the AGV has reached or is currently on
lastNodeSequenceId	integer	The sequence number of the last reached node. Essential for distinguishing nodes visited multiple times in a loop
driving	boolean	True if the AGV is moving/rotating, False if stopped.
batteryState	object	Contains batteryCharge (SoC %) and charging status
agvPosition	object	Current coordinates: $\{x, y, \theta, mapId\}$
actionStates	array[object]	List of states for current actions (WAITING, RUNNING, FINISHED)

3.2.3 Communication Sequence Logic

The interaction follows an event-driven logic to ensure state consistency:

1. Order Dispatch: The Server publishes a JSON message to the .../order topic.
2. Acknowledgment: The AGV receives the message, parses the path, and immediately publishes a state message reflecting the new orderId.
3. Execution: As the AGV traverses the graph, it updates lastNodeId and nodeStates in the state topic.
4. Completion: Upon reaching the final destination, the AGV reports the task status as FINISHED and awaits the next instruction.

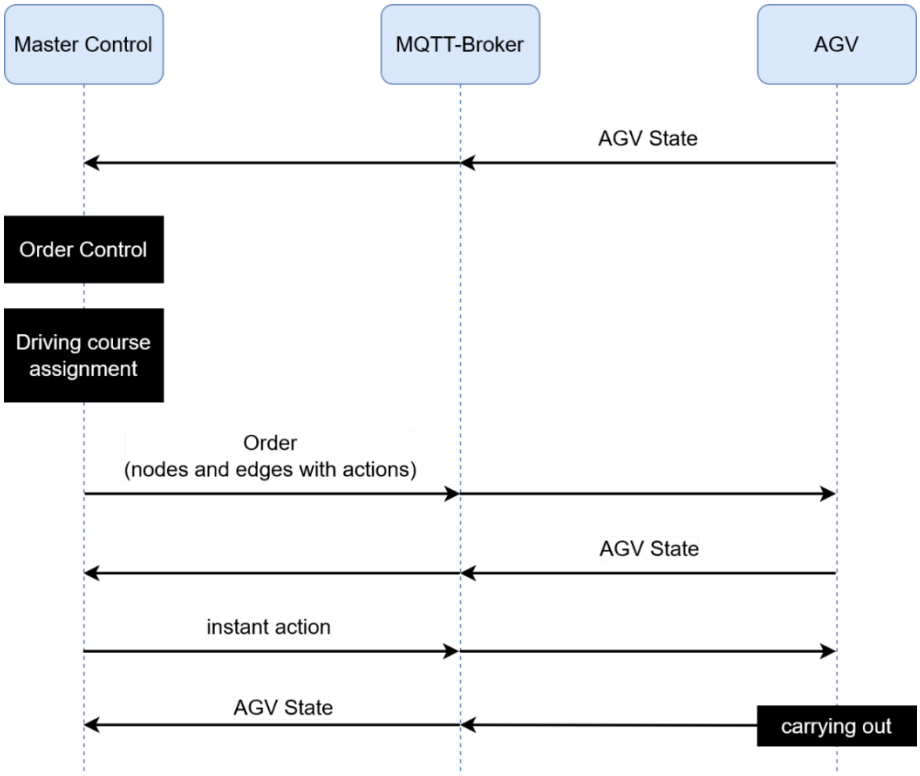


Figure 3.3 Structure of the Information Flow

3.3 Database Design

The data persistence layer is built upon PostgreSQL, an open-source object-relational database system chosen for data integrity, and support for JSON data types. This allows the system to store flexible, schema-less VDA 5050 payloads (like complex order paths or battery states) while maintaining strict relational integrity for core entities like users and vehicles.

The interaction with the database is abstracted through Django ORM (Object-Relational Mapping), which allows for secure and efficient database manipulations using Python classes.

The logical structure of the database is visualized in Figure 3.4. The schema is centered around the AGV entity, which maintains one-to-many relationships with both Order (tasks) and AGV State (telemetry logs). Additionally, the User entity tracks the operator responsible for creating orders.

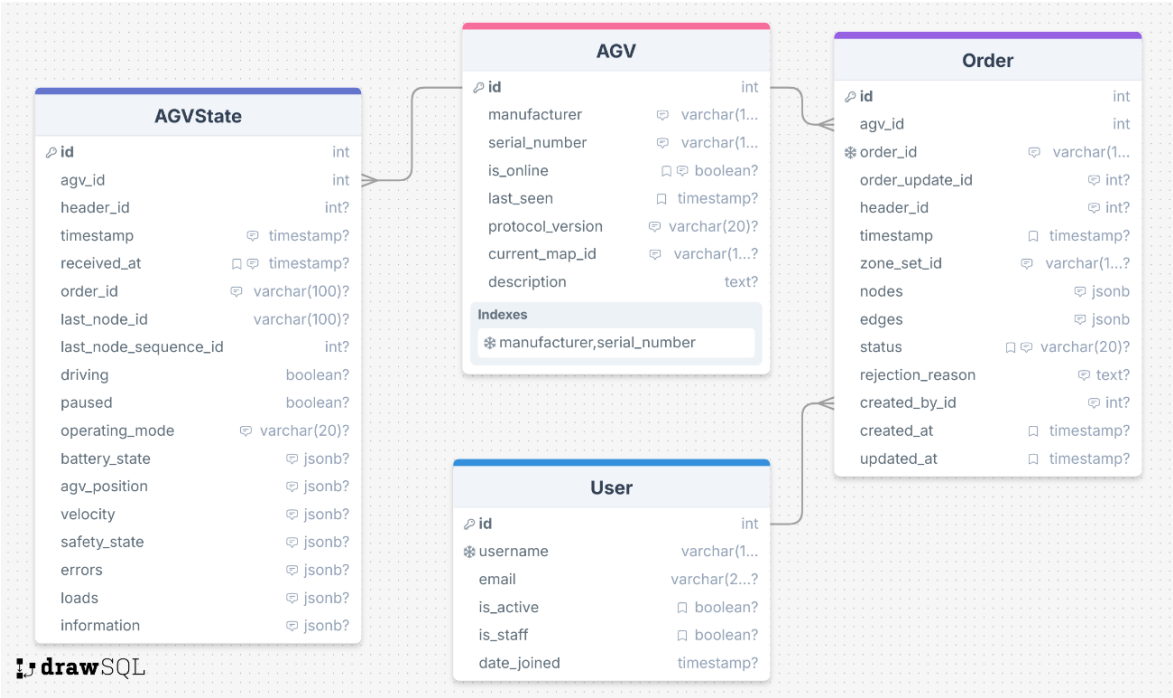


Figure 3.4 Entity-Relationship Diagram of the Master Control System

The following data dictionaries describe the specific fields and data types derived from the system implementation.

a) Fleet Configuration Table (AGV)

This table acts as the central registry for the fleet. It stores the static identification and real-time connectivity status of each robot.

Table 3.5 AGV metadata table

Field Name	Data Type	Description
id	integer (PK)	Auto-increment primary key
manufacturer	varchar	The manufacturer ID
serial_number	varchar	The unique serial number of the AGV
is_online	boolean	Time last message was received from this AGV
last_seen	Timestamp	Stores the battery object
protocol_version	varchar	The VDA 5050 version supported
current_map_id	varchar	Map the AGV is currently operating on

b) Task Management Table (Order)

This table manages the lifecycle of transport missions. It stores the exact topological path sent to the AGV.

Table 3.6 Order data table

Field Name	Data Type	Description
id	integer (PK)	Internal database ID
order_id	varchar	The unique UUID of the order
agv_id	FK (AGV)	Foreign Key linking to the assigned vehicle

created_by_id	FK (User)	Foreign Key linking to the operator who initiated the task
nodes	JSONB	A JSON array containing the list of nodes (Base and Horizon) for the path
edges	JSONB	A JSON array containing the list of edges connecting the nodes
status	varchar	Current state
order_update_id	integer	Sequence number for order extensions (starts at 0)

c) Telemetry Log Table (AGVState)

Designed as a time-series log, this table captures the high-frequency status reports sent by AGVs via the state topic. It utilizes JSONB columns to store nested VDA 5050 objects flexibly.

Table 3.7 AGV States table

Field Name	Data Type	Description
id	integer (PK)	Auto-increment ID
agv_id	FK (AGV)	Reference to the reporting AGV
timestamp	Timestamp	Time when the state was generated by the AGV
driving	boolean	Operational status
battery_state	JSONB	Stores the battery object
agv_position	JSONB	Stores position data
velocity	JSONB	Stores velocity vectors
errors	JSONB	Array of active error objects
last_node_id	Varchar	The ID of the last node the AGV traversed

d) User Management Table (User)

This table utilizes Django's built-in authentication system to manage system access.

Table 3.8 User table

Field Name	Data Type	Description
id	integer (PK)	Auto-increment primary key
username	varchar	Unique username for login
is_staff	boolean	Determines if the user can access the admin interface
date_joined	Timestamp	Account creation time

3.4 Software Module Design

The core logic of the Master Control Server is implemented using Python, adhering to Object-Oriented Programming (OOP) principles to ensure modularity

and reusability. The codebase is organized into distinct service modules, each responsible for a specific domain of the VDA 5050 control loop.

The interaction between the software modules is illustrated in the UML Class Diagram below. The architecture follows a hierarchical structure where the TaskManager acts as the orchestrator, delegating specific computational tasks to specialized engines.

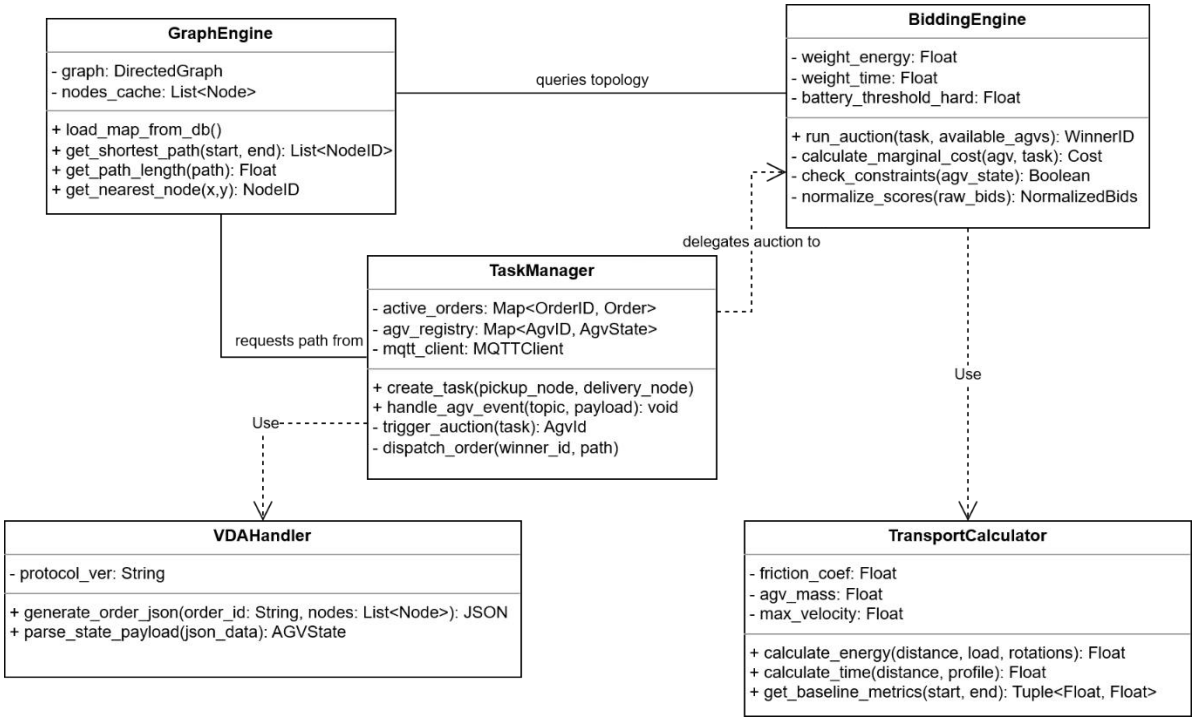


Figure 3.5 UML Class Diagram

3.4.1 Task Manager Module

This module serves as the central controller of the application. It acts as the bridge between the external REST API requests and the internal algorithmic processing.

- Handles the creation, queuing, assignment, and completion of transport orders.
- Serializes the logical path into the VDA 5050 JSON format (Nodes/Edges generation) before dispatching to the MQTT broker.
- Listens for finished events from AGVs to trigger the next order in the queue (Task Chaining execution).

Key methods:

- create_task(pickup, delivery): Validates input and initiates the auction process.
- dispatch_order(winner_agv, order_data): Publishes the VDA 5050 payload to the uagv/v2/.../order topic.

3.4.2 Bidding Engine Module

The BiddingEngine encapsulates the SSI Auction Algorithm defined in Section 2.4. It is designed to be stateless, processing snapshots of the fleet's current status to make optimal decisions.

- Computes the cost difference $(J_{new} - J_{old})$ for every available AGV.
- Filters out candidates based on Battery Thresholds (Hard/Soft constraints) and topological reachability.
- Aggregates normalized Energy and Time scores to identify the vehicle with the lowest bid.

Key methods:

- calculate_bid(agv_state, task): Returns the calculated cost for a specific agent.
- run_auction(task, available_agvs): Iterates through the fleet and returns the winner_id.

3.4.3 Graph Engine Module

This module abstracts the complexity of the digital map. It utilizes the NetworkX library to manage the directed graph structure $G(N, E)$.

- Loads node coordinates and edge weights from the database into memory for fast access.
- Implements the Dijkstra algorithm to find the shortest path sequence between any two nodes.
- Provides helper functions to calculate the total distance of a route.

Key methods:

- get_shortest_path(start, end): Returns a list of Node IDs.
- get_path_distance(path): Returns the total length in meters.

3.4.4 Physics Calculation Module

To ensure the auction is based on realistic metrics rather than just distance, this module implements the kinematic models described in Section 2.3.

- Energy Estimation: Calculates energy consumption (kJ) based on friction, load weight, and rotational torque.
- Time Estimation: Calculates travel time (seconds) using a trapezoidal velocity profile (Acceleration → Constant Speed → Deceleration).
- Baseline Computation: Provides the ideal cost used for normalizing bids in the auction process.

Key methods:

- calculate_energy(distance, load, rotation_count): Returns estimated Energy.
- calculate_time(distance, rotation_count): Returns estimated Time.

3.5 Application Programming Interface (API) Design

To facilitate seamless integration with the Web GUI (Frontend) and potential external systems, the Master Control Server exposes a comprehensive set of RESTful APIs. These APIs are built using the Django REST Framework (DRF), adhering to standard HTTP architectural constraints to ensure scalability and stateless communication.

The API design follows these core principles:

- URLs represent resources (e.g., agvs, orders) rather than actions.
- Utilizes HTTP verbs (GET, POST, PUT, DELETE) for CRUD operations.
- All request bodies and response payloads are formatted in JSON.
- Uses standard HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found) to indicate operation results.

The system exposes over 15 endpoints organized into functional groups: Authentication, Fleet Management, Map Topology, and Order Processing. As shown in Figure 3.6, the API structure strictly follows RESTful principles using standard HTTP methods (GET, POST, PUT, DELETE).

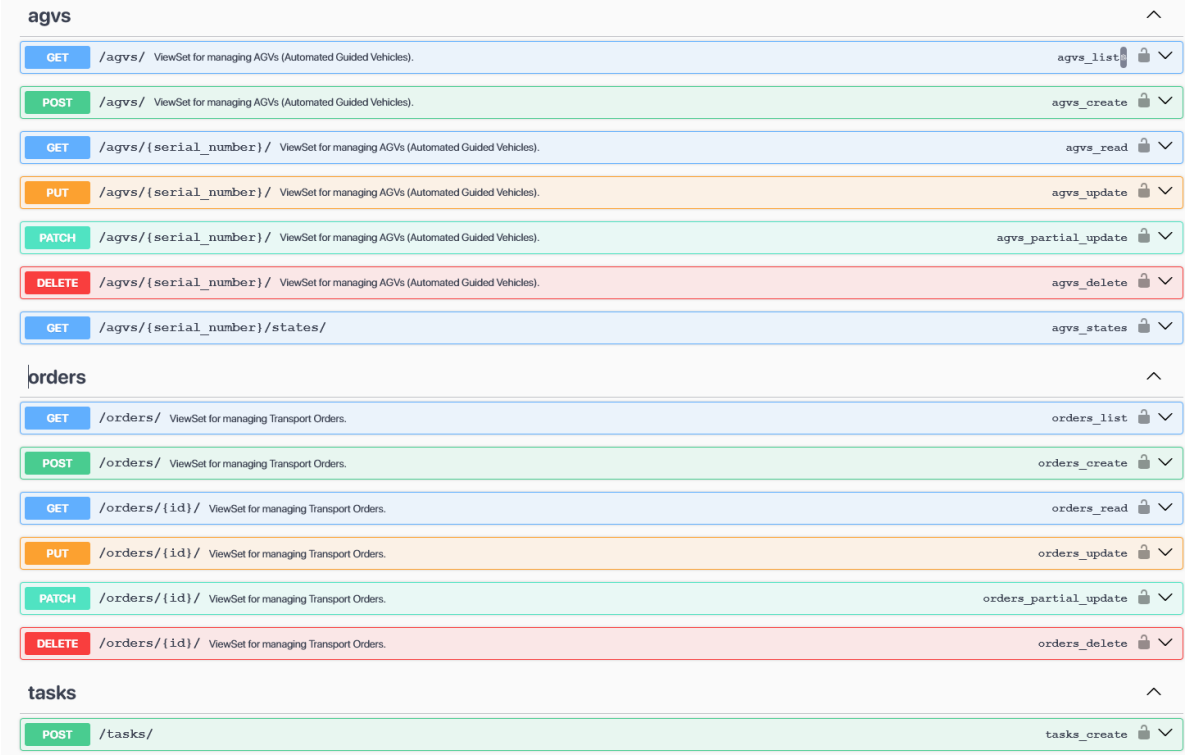
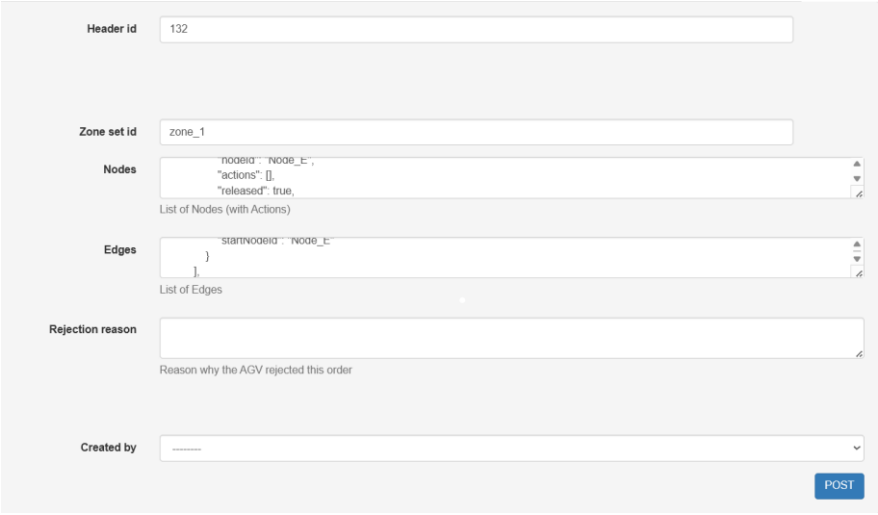


Figure 3.6 Overview of the Master Control System's API endpoints displayed in Swagger UI.

The most critical endpoint in the system is the Task Creation API (POST /api/orders/). This interface acts as the entry point for the logic described in Chapter 2.

Figure 3.7 illustrates the required JSON payload for submitting a transport request.

- Input: The client must provide the pickup_node_id, delivery_node_id, and priority.
- Validation: The backend automatically validates these IDs against the Graph Database to ensure the nodes exist before triggering the Bidding Engine.
- Response: On success (HTTP 200), the server returns a unique order_id which allows the client to track the mission status via WebSocket or polling.




```
{
    "id": 132,
    "header_id": 0,
    "timestamp": "2026-01-12T13:23:18.374361+07:00",
    "order_id": "ORD_0C5056A2",
    "order_update_id": 0,
    "zone_set_id": "zone_1",
    "nodes": [
        {
            "nodeId": "Node_E",
            "actions": [],
            "released": true,
            "sequenceId": 0,
            "nodePosition": {
                "x": 0.0,
                "y": 10.0,
                "mapId": "default_map"
            }
        },
        {
            "nodeId": "Node_A",
            "actions": [],
            "released": true,
            "sequenceId": 2,
            "nodePosition": {
                "x": 0.0,
                "y": 0.0,
                "mapId": "default_map"
            }
        }
    ],
    "edges": [
        {
            "edgeId": "Node_E_Node_A",
            "maxSpeed": 2.0,
            "released": true,
            "endNodeId": "Node_A",
            "sequenceId": 1,
            "startNodeId": "Node_E"
        }
    ],
    "status": "COMPLETED",
    "rejection_reason": null,
    "created_at": "2026-01-12T13:23:18.374399+07:00",
    "updated_at": "2026-01-12T13:23:56.044671+07:00",
    "agv": 67,
    "created_by": null
},
```


Figure 3.7 Detailed schema of the 'Create Order' API. The JSON payload requires specific topological node IDs to initiate the auction process.

CHAPTER 4. EXPERIMENT AND RESULTS

4.1 Experimental Setup

The scalability and stability test was conducted with a larger fleet configuration to evaluate the logic's performance under high-traffic conditions.

- Fleet Configuration: 7 Virtual AGVs (AGV_01 to AGV_07).
- Initial State: AGVs are randomly distributed across Nodes A-H with varying battery levels (65% - 95%).
- Test Scenario: A batch of 10 Transport Tasks was injected consecutively with a 5-second interval between requests.
- Map Topology: A simplified graph consisting of 8 key nodes (Node_A to Node_H) and bidirectional edges.

MANUFACTURER	SERIAL NUMBER	IS ONLINE	LAST SEEN	CURRENT MAP ID
TestManufacturer	AGV_07	✓	Feb. 6, 2026, 2:15 a.m.	-
TestManufacturer	AGV_06	✓	Feb. 6, 2026, 2:15 a.m.	-
TestManufacturer	AGV_05	✓	Feb. 6, 2026, 2:15 a.m.	-
TestManufacturer	AGV_04	✓	Feb. 6, 2026, 2:15 a.m.	-
TestManufacturer	AGV_03	✓	Feb. 6, 2026, 2:15 a.m.	-
TestManufacturer	AGV_02	✓	Feb. 6, 2026, 2:15 a.m.	-
TestManufacturer	AGV_01	✓	Feb. 6, 2026, 2:15 a.m.	-

```
============================================================
Configuration:
  - MQTT Broker: 127.0.0.1:1884
  - Number of AGVs: 7
  - Idle drain: 0.05%/s
  - Moving drain: 0.15%/s
  - Moving with load drain: 0.25%/s
============================================================

🤖 Initialized AGV_01 at Node_B (Battery: 95%)
🤖 Initialized AGV_02 at Node_C (Battery: 90%)
🤖 Initialized AGV_03 at Node_D (Battery: 85%)
🤖 Initialized AGV_04 at Node_A (Battery: 80%)
🤖 Initialized AGV_05 at Node_B (Battery: 75%)
🤖 Initialized AGV_06 at Node_C (Battery: 70%)
🤖 Initialized AGV_07 at Node_D (Battery: 65%)
```

Figure 4.1 List of simulation AGVs and its initial states

4.2 Functional Verification

The system logs confirm that the Bidding Engine successfully processed all 10 requests without failure.

4.2.1 Task Assignment Logic

The logs demonstrate the system's ability to identify the optimal vehicle based on the lowest Marginal Cost.

- Scenario (Task #1): Request from Node_A to Node_C.

- Decision: Although AGV_04 was at Node_E (close), AGV_02 was selected because its trajectory allowed for a smoother path integration, resulting in the lowest bid score.

```
📦 AGV_02: Received Order ORD_41866634
   Path: ['Node_F', 'Node_B', 'Node_A', 'Node_B', 'Node_C']
```

Figure 4.2 Log snippet showing successful winner determination

4.2.2 Task Chaining Capability

A key feature of the proposed algorithm is handling busy vehicles. The logs show that AGV_01 was assigned 3 consecutive tasks. Instead of waiting for the vehicle to become IDLE, the system successfully queued the new orders (ORD_B370..., ORD_E83C..., ORD_9EC8...) based on the predicted future location of AGV_01.

ORDER ID	AGV	STATUS	CREATED AT	TIMESTAMP
ORD_924AEDFC	TestManufacturer - AGV_07	Completed	Feb. 6, 2026, 2:24 a.m.	Feb. 6, 2026, 2:24 a.m.
ORD_3A1447A8	TestManufacturer - AGV_05	Queued for Sending	Feb. 6, 2026, 2:23 a.m.	Feb. 6, 2026, 2:23 a.m.
ORD_FA9D2D8B	TestManufacturer - AGV_04	Completed	Feb. 6, 2026, 2:23 a.m.	Feb. 6, 2026, 2:23 a.m.
ORD_9EC82730	TestManufacturer - AGV_01	Completed	Feb. 6, 2026, 2:23 a.m.	Feb. 6, 2026, 2:23 a.m.
ORD_ECD7C505	TestManufacturer - AGV_04	Completed	Feb. 6, 2026, 2:23 a.m.	Feb. 6, 2026, 2:23 a.m.
ORD_E83C8793	TestManufacturer - AGV_01	Completed	Feb. 6, 2026, 2:23 a.m.	Feb. 6, 2026, 2:23 a.m.
ORD_12936102	TestManufacturer - AGV_07	Completed	Feb. 6, 2026, 2:23 a.m.	Feb. 6, 2026, 2:23 a.m.
ORD_81B45A0B	TestManufacturer - AGV_02	Completed	Feb. 6, 2026, 2:23 a.m.	Feb. 6, 2026, 2:23 a.m.
ORD_B370F79B	TestManufacturer - AGV_01	Completed	Feb. 6, 2026, 2:23 a.m.	Feb. 6, 2026, 2:23 a.m.

Figure 4.3 List of simulation Orders

4.3 Performance Evaluation

The quantitative analysis is based on the execution metrics of the 10-task batch.

4.3.1 Load Balancing

Unlike a Round-Robin scheduler which forces equality, the SSI Auction Algorithm prioritizes global efficiency. As shown in Table 4.1, the workload was distributed based on cost efficiency:

- AGV_01 (3 tasks) and AGV_02 (2 tasks) handled 50% of the workload due to their central positioning (Node_B, Node_C).
- AGV_03 and AGV_06 remained idle. This indicates that activating these vehicles would have incurred a higher "Deadhead" cost (traveling empty to pickup) compared to reusing the active vehicles.

Table 4.1 Task distribution among the fleet

ID	Number of Tasks	Percentage
AGV_01	3	30%
AGV_02	2	20%
AGV_03	0	0%
AGV_04	2	20%
AGV_05	1	10%

AGV_06	0	0%
AGV_07	2	20%

```
AGV_01:  3 tasks (9 edges)
AGV_02:  2 tasks (7 edges)
AGV_04:  2 tasks (5 edges)
AGV_05:  1 tasks (1 edges)
AGV_07:  2 tasks (6 edges)
```

Figure 4.4 Log of tasks each AGV did in the simulation

4.3.2 Execution Efficiency

- Total Execution Time: 66.53 seconds for 10 tasks.
- Average Processing Time: 6.65 seconds per task.
- System Throughput: 0.15 Tasks/second. These figures confirm that the Python-based backend is capable of handling high-frequency requests with negligible latency.

4.4 Discussion

The test results highlight a trade-off between Efficiency and Equality. While mathematically optimal for energy and time (minimizing total distance traveled: 28 edges), this strategy may lead to uneven wear-and-tear on specific vehicles (e.g., AGV_01).

CHAPTER 5. CONCLUSION

5.1 Conclusion

This thesis presented the design and implementation of a centralized Master Control System for AGV fleets, complying with the industrial VDA 5050 standard. The research addressed the critical challenge of multi-agent task allocation by proposing and verifying the SSI (Sequential Single-Item) Auction Algorithm.

The primary objectives set out have been achieved:

- Successfully developed a scalable 3-tier architecture (User - Server - Agent) using Django, MQTT, and Docker. The system ensures robust communication via the VDA 5050 protocol, allowing for standardized interoperability between the control layer and heterogeneous vehicles.
- The proposed Marginal Cost-based Auction mechanism was implemented to solve the dispatching problem. Unlike simple distance-based heuristics, this approach integrates multiple variables – Energy Consumption, Time, and Battery constraints – into a hybrid objective function.
- The system demonstrated the ability to queue tasks for busy vehicles (Task Chaining), optimizing fleet utilization by predicting future AGV locations rather than relying solely on currently idle agents.

5.2 Future Work

To address the aforementioned limitations and enhance the system's applicability in industrial environments, future research will focus on the following areas:

- To improve load balancing, the cost function will be modified to include a "Fatigue Penalty" or "Usage History Factor". This would discourage the system from assigning consecutive tasks to the same vehicle, ensuring a more uniform distribution of workload and battery cycles across the fleet.
- Implementing a Reservation-based Traffic Control (based on VDA 5050's Node/Edge reservation). The server will manage a time-space map to prevent AGVs from entering the same node simultaneously, proactively preventing deadlocks.
- Transitioning from simulation to a physical testbed using real mobile robots (e.g., ROS-based robots). This will allow for the validation of real-world communication latency and physical dynamics.
- Integrating a mechanism to handle execution failures. If an AGV reports a "Blocked" error or falls offline (MQTT Last Will), the Task Manager should automatically trigger a Re-auction process to reassign the pending task to another available agent immediately.

REFERENCES

[1] K. Ellithy, M. Salah, I. S. Fahim, and R. Shalaby, "AGV and Industry 4.0 in warehouses: a comprehensive analysis of existing literature and an innovative framework for flexible automation," *Int. J. Adv. Manuf. Technol.*, vol. 134, no. 1–2, pp. 15–38, Sep. 2024, doi: 10.1007/s00170-024-14127-0.

[2] J. Theunissen, H. Xu, R. Y. Zhong, and X. Xu, "Smart AGV System for Manufacturing Shopfloor in the Context of Industry 4.0," in *2018 25th International Conference on Mechatronics and Machine Vision in Practice (M2VIP)*, Stuttgart: IEEE, Nov. 2018, pp. 1–6. doi: 10.1109/M2VIP.2018.8600887.

[3] L. Qiu, W.-J. Hsu, S.-Y. Huang, and H. Wang, "Scheduling and routing algorithms for AGVs: A survey," *Int. J. Prod. Res.*, vol. 40, no. 3, pp. 745–760, Jan. 2002, doi: 10.1080/00207540110091712.

[4] I. F. A. Vis, "Survey of research in the design and control of automated guided vehicle systems," *Eur. J. Oper. Res.*, vol. 170, no. 3, pp. 677–709, May 2006, doi: 10.1016/j.ejor.2004.09.020.

[5] Verband der Automobilindustrie e.V. (VDA), "VDA 5050: Interface for the communication between automated guided vehicles (AGVs) and a master control," vol. Version 2.1.0, Jan. 2025, [Online]. Available: https://www.vda.de/dam/jcr:f0c9c019-1506-4dee-998a-e92723fbf025/EN-VDA5050-V2_0_0.pdf

[6] M. De Ryck, D. Pissoort, T. Holvoet, and E. Demeester, "Decentral task allocation for industrial AGV-systems with routing constraints," *J. Manuf. Syst.*, vol. 62, pp. 135–144, Jan. 2022, doi: 10.1016/j.jmsy.2021.11.012.

[7] H. T. Dinh, "Multi-Agent Route Planning Using Delegate MAS".

[8] L. Ďurica, M. Gregor, V. Vavrík, M. Marschall, P. Grznár, and Š. Mozol, "A Route Planner Using a Delegate Multi-Agent System for a Modular Manufacturing Line: Proof of Concept," *Appl. Sci.*, vol. 9, no. 21, p. 4515, Oct. 2019, doi: 10.3390/app9214515.

[9] B. Micieta *et al.*, "Delegate MASs for coordination and control of one-directional AGV systems: a proof-of-concept," *Int. J. Adv. Manuf. Technol.*, vol. 94, no. 1–4, pp. 415–431, Jan. 2018, doi: 10.1007/s00170-017-0915-8.

[10] T. Holvoet and P. Valckenaers, "Exploiting the Environment for Coordinating Agent Intentions," in *Environments for Multi-Agent Systems III*, vol. 4389, D. Weyns, H. V. D. Parunak, and F. Michel, Eds., in Lecture Notes in Computer Science, vol. 4389. , Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 51–66. doi: 10.1007/978-3-540-71103-2_3.

[11] P. Valckenaers and H. Van Brussel, "Holonic Manufacturing Execution Systems," *CIRP Ann.*, vol. 54, no. 1, pp. 427–432, 2005, doi: 10.1016/S0007-8506(07)60137-1.

[12] H. A. Nguyen, D. M. Nguyen, Q. H. Pham, Q. D. Pham, and D. C. Hoang, "Energy and Time-Efficient Scheduling of Automated Guided Vehicles System: A Hybrid Artificial Bee Colony Algorithm and Improved Ant Colony Optimization Approach," in *2023 12th International Conference on Control, Automation and Information Sciences (ICCAIS)*, Hanoi, Vietnam: IEEE, Nov. 2023, pp. 719–724. doi: 10.1109/ICCAIS59597.2023.10382354.

[13] D. Rivas and L. Ribas-Xirgo, "Agent-based Model for Transport Order Assignment in AGV Systems," in *2019 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, Zaragoza, Spain: IEEE, Sep. 2019, pp. 947–954. doi: 10.1109/ETFA.2019.8869302.

[14] M. De Ryck, D. Pissoort, T. Holvoet, and E. Demeester, "Decentral task allocation for industrial AGV-systems with resource constraints," *J. Manuf. Syst.*, vol. 59, pp. 310–319, Apr. 2021, doi: 10.1016/j.jmsy.2021.03.008.

[15] N. Sullivan, S. Grainger, and B. Cazzolato, "Sequential single-item auction improvements for heterogeneous multi-robot routing," *Robot. Auton. Syst.*, vol. 115, pp. 130–142, May 2019, doi: 10.1016/j.robot.2019.02.016.

[16] D. Weyns, T. Holvoet, K. Schelfthout, and J. Wielemans, "Decentralized control of automatic guided vehicles: applying multi-agent systems in practice," in *Companion to the 23rd ACM SIGPLAN conference on Object-oriented programming systems languages and applications*, Nashville TN USA: ACM, Oct. 2008, pp. 663–674. doi: 10.1145/1449814.1449819.